Chapitre 02 — Les variables

Objectif Mémoriser une valeur, l'afficher dans une phrase et demander une réponse à l'utilisateur.

1. Une variable donne un nom à une valeur

```
prenom="Mila"
echo "Bonjour $prenom"
```

Résultat : `Bonjour Mila`.

La première ligne **affecte** une valeur : Bash range `Mila` et retient le nom `prenom`. La seconde **lit** cette valeur avec `$prenom` : Bash remplace le nom par la valeur, puis `echo` affiche la phrase obtenue.

Une valeur, un nom, une utilisation



De l'affectation à l'affichage

Écriture	Ce qu'elle signifie
`prenom="Mila"`	Ranger `Mila` dans la variable `prenom`
`$prenom`	Demander la valeur rangée sous ce nom
`"$prenom"`	Utiliser cette valeur comme un seul argument

Le `$` est la marque de la **lecture**. On ne l'écrit jamais au moment d'affecter.

Aucun espace autour de `=` `prenom="Mila"` fonctionne. `prenom = "Mila"` échoue : Bash y voit une commande nommée `prenom`, à laquelle on passerait les arguments `=` et `Mila`.

```
prenom : commande introuvable
```

C'est l'erreur la plus fréquente du chapitre. Le `=` se colle des deux côtés.

Choisis des noms lisibles : `prenom`, `nom_station`, `nombre_messages`. Un nom ne commence pas par un chiffre. `prenom` et `PRENOM` sont deux variables différentes.

2. Les guillemets doubles gardent la valeur en un seul morceau

Après avoir remplacé `$station` par sa valeur, Bash **découpe le résultat en mots** à chaque espace — sauf si la lecture est entre guillemets doubles.

```
station="Orion    Nord"
echo $station
echo "$station"
```

Résultats, dans cet ordre :

```
Orion Nord
Orion    Nord
```

Sans guillemets, Bash a coupé la valeur en deux arguments, `Orion` et `Nord`, et `echo` les a recollés avec **un seul** espace. Les espaces d'origine ont disparu : c'est la preuve visible que la valeur a été découpée.

Avec guillemets, `echo` reçoit **un seul** argument, qui contient la valeur exacte.

Tant qu'on affiche du texte, le dégât reste cosmétique. Il devient réel dès qu'on manipule un fichier :

```
fichier="rapport final.txt"
cat $fichier      # cat cherche DEUX fichiers : « rapport » puis « final.txt »
cat "$fichier"    # cat cherche LE fichier : « rapport final.txt »
```

> Habitude à prendre Quand tu utilises une variable comme argument, écris `"$variable"`. Les guillemets ne coûtent rien quand la valeur n'a pas d'espace, et sauvent la commande quand elle en a.

3. Guillemets simples ou guillemets doubles ?

Les deux protègent les espaces. Ils se distinguent sur un seul point : le `$`.

```
station="Orion Nord"
echo "$station"
echo '$station'
```

Résultats, dans cet ordre :

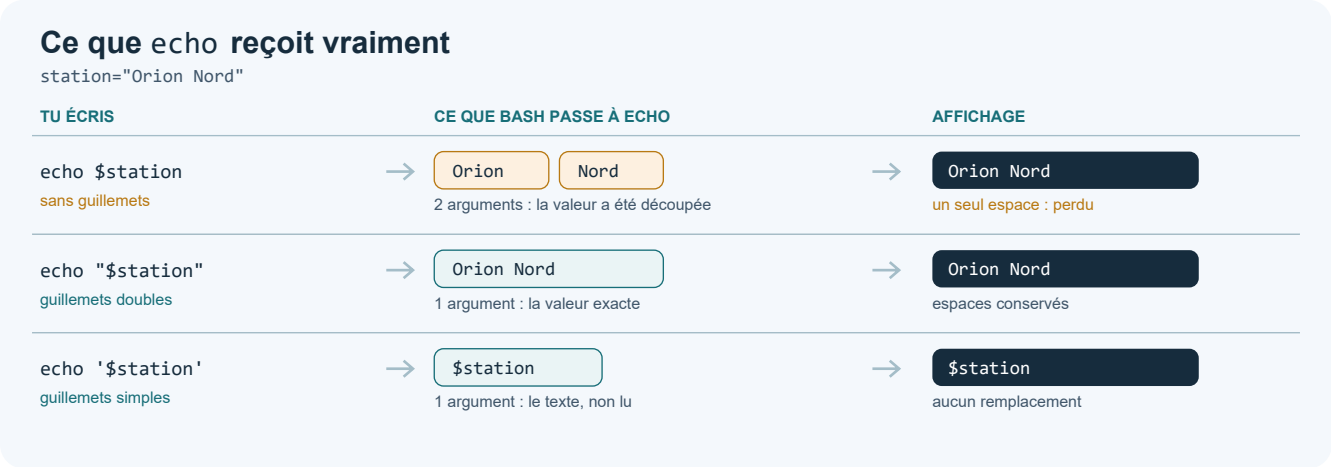
```
Orion Nord
$station
```

- Les **guillemets doubles** laissent Bash lire la variable, tout en gardant sa valeur groupée.
- Les **guillemets simples** conservent le texte tel quel : aucun `$` n'est interprété.

Tu veux	Écris
La valeur de la variable	`"$station"`
Le texte `$station` lui-même	`'$station'`
Un texte à transmettre tel quel à un autre programme	`'echo $station'`

La troisième ligne servira au §9 : elle permet d'envoyer un `$` à un autre shell sans que le tien y touche au passage.

Les trois écritures des §2 et §3, côte à côte :



Ce que echo reçoit vraiment selon les guillemets

4. Les accolades : délimiter le nom, prévoir un défaut

Délimiter le nom

```
mission="orion"
fichier="${mission}_rapport.txt"
echo "$fichier"
```

Résultat : `orion_rapport.txt`.

Les accolades disent où le nom s'arrête. Sans elles, `"$mission_rapport.txt"` fait chercher à Bash une variable nommée `mission_rapport` — le souligné fait partie des caractères autorisés dans un nom. Cette variable n'existe pas, donc Bash la remplace par du vide : il ne reste que `.txt`.

Prévoir une valeur par défaut

```
surnom=""
echo "Nom affiche : ${surnom:-visiteur}"
surnom="Mila"
echo "Nom affiche : ${surnom:-visiteur}"
```

Résultats, dans cet ordre :

```
Nom affiche : visiteur
Nom affiche : Mila
```

`${surnom:-visiteur}` fournit `visiteur` **seulement** si `surnom` est vide ou absente. La variable n'est pas remplie pour autant : après le premier affichage, `surnom` est toujours vide.

5. Poser une question avec read

```
read -p 'Ton prenom : ' prenom
echo "Bienvenue $prenom"
```

- `read` attend une ligne et la range dans `prenom`.

- `-p` affiche la question dans le terminal.

Tape une réponse, puis appuie sur **Entrée**. Le nom après la question est celui de la variable : on écrit `prenom`, sans `$`, parce que c'est une affectation.

Avec une seule variable, `read` range **toute la ligne**, espaces compris : une réponse comme `Mila Keller` arrive entière dans `prenom`. Raison de plus pour la relire avec `"$prenom"`.

Ces lignes s'exécutent dans ton terminal Bash. Le programme attend réellement ta saisie : ce n'est pas un blocage.

6. Garder le résultat d'une commande

```
machine=$(hostname)
jour=$(date +%F)
echo "Machine : $machine — date : $jour"
```

`$(...)` exécute la commande et récupère sa sortie. Ici, Bash garde le nom de la machine et la date au format année-mois-jour. La valeur de la date dépend du jour où tu exécutes l'exemple.

Ne confonds pas : `"$machine"` lit une variable, `$(hostname)` lance une commande.

7. Calculer avec des nombres entiers

```
batteries=4
energie_par_batterie=25
energie_totale=$((batteries * energie_par_batterie))
echo "Energie disponible : $energie_totale unites"
```

Résultat : `Energie disponible : 100 unites`.

Une variable Bash contient toujours du **texte** : `batteries` contient le texte `4`, pas un nombre. `$((...))` demande justement à Bash de lire ce texte comme un nombre et de calculer avec. C'est la raison d'être des doubles parenthèses.

À l'intérieur, le `$` de lecture devient facultatif : `$((batteries * 25))` et `$(($batteries * 25))` donnent le même résultat. La forme sans `$` est la plus courante.

Opérateur	Calcul
`+`, `-`	Addition, soustraction
`*`	Multiplication
`/`	Division entière : `7 / 2` donne `3`
`%`	Reste : `7 % 2` donne `1`

Nous utilisons ici des valeurs fixées dans le cours. Avant de calculer avec une saisie libre, il faudra vérifier qu'elle contient bien un nombre : ce sera un défi du chapitre 04.

8. Les variables déjà définies par le système

Ton shell contient déjà des variables que tu n'as pas créées :

```
echo "Dossier personnel : $HOME"
echo "Recherche des commandes : $PATH"
```

`HOME` désigne ton dossier personnel. `PATH` liste les dossiers où le shell recherche les commandes quand leur nom ne contient pas de `/`. La commande `env` affiche toutes les variables de cette famille ; nous y revenons au §9.

Elles se lisent exactement comme les tiennes, avec `$`. Par convention, ces variables du système s'écrivent en MAJUSCULES : garde les minuscules pour les tiennes, tu éviteras d'en écraser une par accident.

9. Transmettre une variable à un autre programme : export

Une variable ordinaire n'existe que dans le shell qui l'a créée. `export` la rend disponible aux programmes lancés **ensuite** par ce shell.

```
export NOM_STATION="Orion"
bash -c 'echo "Station recue : [$NOM_STATION]"'
```

Résultat : `Station recue : [Orion]`. Le second Bash est un processus enfant, comme au chapitre 01 ; il reçoit `NOM_STATION` grâce à `export`.

La contre-épreuve, sans `export` :

```
PAS_EXPORTEE="Vega"
bash -c 'echo "Non exportee : [$PAS_EXPORTEE]"'
```

Résultat : `Non exportee : []`. Les crochets sont vides : l'enfant n'a jamais reçu la variable. C'est précisément ce que `export` corrige.

> Les guillemets simples de `bash -c '...'` sont indispensables Ils empêchent **ton** shell de remplacer `$PAS_EXPORTEE` avant de lancer l'enfant. Avec des guillemets doubles, tu verrais s'afficher `[Vega]` — mais la valeur viendrait de ton propre shell, pas de l'enfant : la démonstration serait fausse alors que le résultat a l'air juste. C'est le §3 appliqué à un cas réel.

Voir quelles variables existent

Deux commandes listent les variables de ton shell, et la différence entre les deux est précisément celle de ce paragraphe :

Commande	Ce qu'elle liste
`env`	Uniquement les variables **exportées**, c'est-à-dire l'environnement
`set`	**Toutes** les variables du shell, exportées ou non

`set` donne donc toujours une liste plus longue que `env` : elle contient en plus les variables ordinaires, les tiennes. Essayons avec une de chaque :

```
locale_seule="Vega"
export EXPORTEE="Orion"
```

Les deux listes défilent sur des dizaines de lignes : on cherche un nom précis avec `grep`, le filtre vu au chapitre 01.

```
env | grep EXPORTEE         # trouvée : elle est dans l'environnement
env | grep locale_seule     # rien : elle n'y est pas
set | grep locale_seule     # trouvée : elle existe bien dans ton shell
```

Une variable ordinaire existe donc pour ton shell, mais pas pour l'environnement qu'il transmet à ses enfants. C'est la même constatation qu'avec `bash -c`, obtenue cette fois sans lancer de processus.

Portée et durée de vie. Une variable disparaît avec le processus qui la contient : rien de ce que tu définis ici ne survit à la fermeture du terminal. `export` transmet vers les enfants, jamais vers le parent — aucun script ne peut donc modifier les variables de ton shell. Ces exemples ne touchent à aucun fichier de configuration.

Pour aller plus loin : printf

Pour les messages simples de nos premiers exercices, `echo "Bonjour $prenom"` suffit. `printf` devient utile quand on veut maîtriser le format d'une sortie :

```
printf 'Bonjour %s\n' "$prenom"
```

- Le premier argument est le **format** : `Bonjour %s\n`.
- `%s` reçoit le texte de l'argument suivant, ici `"$prenom"`.
- `\n` ajoute un retour à la ligne. `printf` ne l'ajoute pas automatiquement.

Cette commande donne ici la même phrase qu'avec `echo`. Mais `printf` montre aussi où commence et où finit chaque argument, ce qui rend le §2 enfin lisible :

```
station="Orion Nord"
printf '[%s]\n' "$station"
printf '[%s]\n' $station
```

```
[Orion Nord]
[Orion]
[Nord]
```

Un argument d'un côté, deux de l'autre : le découpage en mots devient visible.

Enfin, `printf '%s\n' "$variable"` traite comme du texte une valeur telle que `-n` ou contenant des antislashs, sans la confondre avec une option ou un échappement de `echo`.

> Un repère pour la suite Utilise `echo` pour une phrase simple. Utilise `printf` pour un format précis, plusieurs lignes ou des données dont tu ne maîtrises pas le contenu. Garde le format fixe et passe les variables dans les arguments suivants.

À retenir

- Affecter : `prenom="Mila"`, sans espace autour du `=`. Lire : `"$prenom"`.
- Les guillemets **doubles** laissent lire le `$` et gardent la valeur en un seul argument ; les **simples** conservent le texte tel quel.
- `${nom}` délimite le nom ; `${nom:-defaut}` fournit une valeur de secours sans remplir la variable.
- `read` récupère une saisie, `$(...)` la sortie d'une commande, `$((...))` le résultat d'un calcul entier.
- Une variable contient toujours du texte ; `$((...))` demande à Bash de le lire comme un nombre.
- `export` transmet une variable aux programmes lancés ensuite ; sans lui, l'enfant reçoit une valeur vide.
- `env` liste les variables exportées, `set` les liste toutes : la différence, ce sont les tiennes.
- `printf` prend le relais de `echo` quand le format compte.

À toi : que produit `printf '%s\n' '$prenom'` ? Et si tu remplaces les guillemets simples par des doubles ? Que contient `fichier` après `mission="orion"` puis `fichier="$mission_rapport.txt"`, et pourquoi ?

Passer au TP